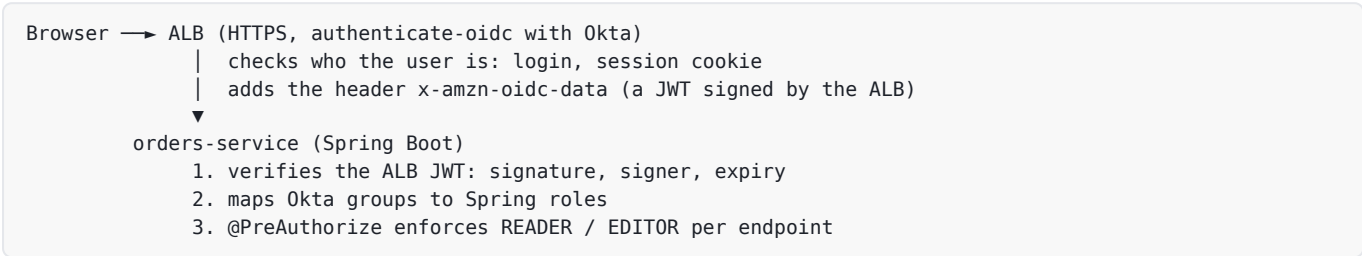


ALB + Okta RBAC for a Spring Boot Microservice

An orders-service with read and edit endpoints. Okta handles sign-in at the ALB, and roles are enforced inside Spring Boot.

How it works



The ALB only checks **who** the user is. It can't allow or block a request based on the user's Okta groups, so **all role checks happen in the service**.

OKTA GROUP	SPRING ROLE	GET /API/ORDERS	PUT /API/ORDERS/{ID}
orders-readonly	ROLE_READER	✓ allowed	✗ 403
orders-editor	ROLE_EDITOR	✓ allowed	✓ allowed
no mapped group	none	✗ 403	✗ 403

Step 1: Okta setup

1a. Create the groups. In **Directory → Groups**, create `orders-readonly` and `orders-editor`, then add users to them.

1b. Create the app. In **Applications → Create App Integration**, choose OIDC and then Web Application. Set the redirect URI to `https://orders.example.com/oauth2/idpresponse`. Assign both groups to the app.

1c. Add groups to the user's claims. Go to **Security → API → Authorization Servers → default → Claims → Add Claim**:

FIELD	VALUE
Name	groups
Include in token type	ID Token, Always
Value type	Groups
Filter	Starts with <code>orders-</code>
Include in	Any scope

The filter keeps the token small and stops you sending every group the user belongs to. The ALB reads claims from Okta's userinfo endpoint, which returns these ID-token claims.

Step 2: ALB listener rule

Set this up as in the earlier guide: an HTTPS listener with an `authenticate-oidc` action followed by a `forward` action. Two settings matter for this service:

```
authenticate_oidc {  
  issuer = "https://your-org.okta.com/oauth2/default"  
  # ... endpoints, client_id, client_secret ...  
  scope = "openid profile email"  
  session_timeout = 3600  
  on_unauthenticated_request = "authenticate" # "deny" for API-only clients  
}
```

Also note the **ALB ARN**, because the service checks for it. You'll also need to allow **outbound 443** from the ALB's security group so it can reach Okta.

Step 3: Spring Boot service

pom.xml

```
<dependencies>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-web</artifactId>  
  </dependency>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-security</artifactId>  
  </dependency>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-actuator</artifactId>  
  </dependency>  
  <dependency>  
    <groupId>com.nimbusds</groupId>  
    <artifactId>nimbus-jose-jwt</artifactId>    <!-- version managed by Spring Boot -->  
  </dependency>  
  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-test</artifactId>  
    <scope>test</scope>  
  </dependency>  
  <dependency>  
    <groupId>org.springframework.security</groupId>  
    <artifactId>spring-security-test</artifactId>  
    <scope>test</scope>  
  </dependency>  
</dependencies>
```

application.yml

```
alb:  
  region: ap-south-1  
  expected-signer: arn:aws:elasticloadbalancing:ap-south-1:123456789012:loadbalancer/app/orders-  
alb/abc123def456  
group-roles:  
  orders-readonly: READER  
  orders-editor: EDITOR
```

AlbProperties.java

```
package com.example.orders.security;
```

```
import org.springframework.boot.context.properties.ConfigurationProperties;
import java.util.Map;

@ConfigurationProperties(prefix = "alb")
public record AlbProperties(String region, String expectedSigner, Map<String, String> groupRoles) {}
```

AlbTokenVerifier.java: validates the ALB-signed JWT

```
package com.example.orders.security;

import com.nimbusds.jose.JWSAlgorithm;
import com.nimbusds.jose.JWSHeader;
import com.nimbusds.jose.crypto.ECDSAVerifier;
import com.nimbusds.jwt.JWTClaimsSet;
import com.nimbusds.jwt.SignedJWT;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.stereotype.Component;

import java.net.URI;
import java.net.http.*;
import java.security.KeyFactory;
import java.security.interfaces.ECPublicKey;
import java.security.spec.X509EncodedKeySpec;
import java.util.*;

@Component
public class AlbTokenVerifier {

    private static final Pattern KID = Pattern.compile("^[A-Za-z0-9-]{1,64}$");
    private static final long CLOCK_SKEW_MS = 60_000;

    private final AlbProperties props;
    private final HttpClient http = HttpClient.newHttpClient();
    private final Map<String, ECPublicKey> keyCache = new ConcurrentHashMap<>();

    public AlbTokenVerifier(AlbProperties props) { this.props = props; }

    public JWTClaimsSet verify(String token) throws Exception {
        SignedJWT jwt = SignedJWT.parse(token);
        JWSHeader header = jwt.getHeader();

        // 1. Algorithm: ALB always signs with ES256
        if (!JWSAlgorithm.ES256.equals(header.getAlgorithm()))
            throw new BadCredentialsException("Unexpected algorithm");

        // 2. Signer: the token must come from OUR ALB, not any ALB in the region
        if (!props.expectedSigner().equals(header.getCustomParam("signer")))
            throw new BadCredentialsException("Token not signed by expected ALB");

        // 3. Key ID: check the format before using it in a URL
        String kid = header.getKeyID();
        if (kid == null || !KID.matcher(kid).matches())
            throw new BadCredentialsException("Invalid kid");

        // 4. Signature
        ECPublicKey key = keyCache.computeIfAbsent(kid, this::fetchKey);
        if (!jwt.verify(new ECDSAVerifier(key)))
            throw new BadCredentialsException("Invalid signature");

        // 5. Expiry
        JWTClaimsSet claims = jwt.getJWTClaimsSet();
        Date exp = claims.getExpirationTime();
        if (exp == null || exp.getTime() + CLOCK_SKEW_MS < System.currentTimeMillis())
            throw new BadCredentialsException("Token expired");

        return claims;
    }
}
```

```

private ECPublicKey fetchKey(String kid) {
    try {
        URI uri = URI.create("https://public-keys.auth.elb." + props.region() + ".amazonaws.com/" + kid);
        HttpResponse<String> res = http.send(HttpRequest.newBuilder(uri).build(),
                                             HttpResponse.BodyHandlers.ofString());

        if (res.statusCode() != 200)
            throw new IllegalStateException("Key fetch failed: HTTP " + res.statusCode());

        String b64 = res.body()
            .replaceAll("-----(BEGIN|END) PUBLIC KEY-----", "")
            .replaceAll("\\s", "");
        byte[] der = Base64.getDecoder().decode(b64);
        return (ECPublicKey) KeyFactory.getInstance("EC")
            .generatePublic(new X509EncodedKeySpec(der));
    } catch (Exception e) {
        throw new IllegalStateException("Cannot load ALB public key " + kid, e);
    }
}

```

AlbAuthenticationFilter.java: maps groups to roles

```

package com.example.orders.security;

import com.nimbusds.jwt.JWTClaimsSet;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.*;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.preauth.PreAuthenticatedAuthenticationToken;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.*;

public class AlbAuthenticationFilter extends OncePerRequestFilter {

    static final String HEADER = "x-amzn-oidc-data";

    private final AlbTokenVerifier verifier;
    private final Map<String, String> groupRoles;

    public AlbAuthenticationFilter(AlbTokenVerifier verifier, Map<String, String> groupRoles) {
        this.verifier = verifier;
        this.groupRoles = groupRoles == null ? Map.of() : groupRoles;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {

        String token = req.getHeader(HEADER);
        if (token != null) {
            try {
                JWTClaimsSet claims = verifier.verify(token);

                List<String> groups = Optional.ofNullable(claims.getStringListClaim("groups"))
                    .orElse(List.of());

                var authorities = groups.stream()
                    .map(groupRoles::get)
                    .filter(Objects::nonNull)
                    .distinct()
                    .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                    .toList();

                String user = Optional.ofNullable(claims.getStringClaim("email"))
                    .orElse(claims.getSubject());
            }

```

```

        var auth = new PreAuthenticatedAuthenticationToken(user, null, authorities);
        auth.setDetails(claims.getClaims());
        SecurityContextHolder.getContext().setAuthentication(auth);

    } catch (Exception e) {
        SecurityContextHolder.clearContext();
        res.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Invalid ALB token");
        return;
    }
}
chain.doFilter(req, res);
}
}

```

SecurityConfig.java

```

package com.example.orders.security;

import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.*;
import org.springframework.http.HttpStatus;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.AnonymousAuthenticationFilter;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;

@Configuration
@EnableMethodSecurity
@EnableConfigurationProperties(AlbProperties.class)
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, AlbTokenVerifier verifier,
                                   AlbProperties props) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // see the CSRF gotcha below
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .addFilterBefore(new AlbAuthenticationFilter(verifier, props.groupRoles()),
                           AnonymousAuthenticationFilter.class)
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health").permitAll() // ALB health checks send no token
                .anyRequest().authenticated())
            .exceptionHandling(e -> e
                .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)));
        return http.build();
    }
}

```

The filter is created with `new` inside the security config rather than declared as a `@Component`. If it were a `@Component`, Spring Boot would also register it as a regular servlet filter, outside the security chain.

OrderController.java: role checks on each endpoint

```

package com.example.orders.api;

import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.web.bind.annotation.*;

import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

@RestController
@RequestMapping("/api")
public class OrderController {

```

```

public record Order(String id, String item, int quantity) {}

private final Map<String, Order> store = new ConcurrentHashMap<>(Map.of(
    "1", new Order("1", "Laptop", 2),
    "2", new Order("2", "Monitor", 5)));

@GetMapping("/orders")
@PreAuthorize("hasAnyRole('READER','EDITOR')")
public Collection<Order> list() {
    return store.values();
}

@GetMapping("/orders/{id}")
@PreAuthorize("hasAnyRole('READER','EDITOR')")
public ResponseEntity<Order> get(@PathVariable String id) {
    return Optional.ofNullable(store.get(id))
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

@PutMapping("/orders/{id}")
@PreAuthorize("hasRole('EDITOR')")
public Order update(@PathVariable String id, @RequestBody Order order) {
    Order updated = new Order(id, order.item(), order.quantity());
    store.put(id, updated);
    return updated;
}

// Debug endpoint: shows the user and roles the service resolved
@GetMapping("/me")
public Map<String, Object> me(Authentication auth) {
    return Map.of(
        "user", auth.getName(),
        "roles", auth.getAuthorities().stream().map(GrantedAuthority::getAuthority).toList());
}
}

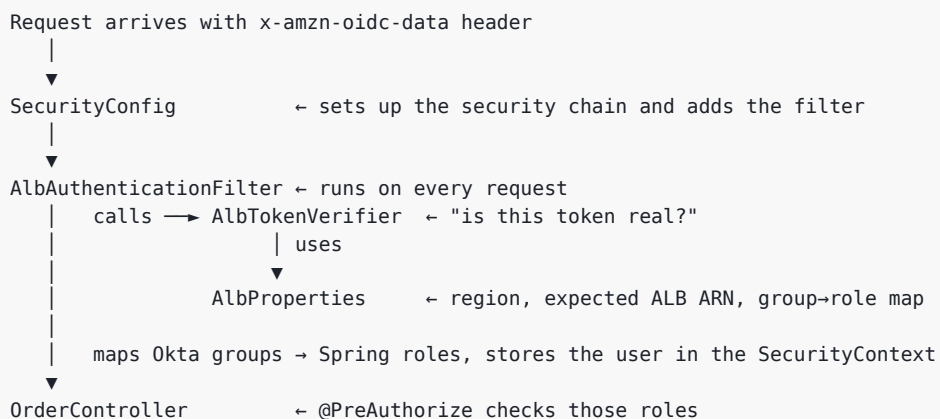
```

What each class does

These four classes work as a pipeline. Each one has a single job:

- **AlbProperties** holds the settings.
- **AlbTokenVerifier** checks that the token is real.
- **AlbAuthenticationFilter** turns the token into a Spring user with roles.
- **SecurityConfig** connects them and sets the rules.

How a request moves through them



AlbProperties: the settings

A plain holder for the values in `application.yml`. It has no logic.

FIELD	USED FOR
<code>region</code>	Building the URL for the ALB's public key, e.g. <code>public-keys.auth.elb.ap-south-1...</code>
<code>expectedSigner</code>	Your ALB's ARN, so the service only accepts tokens signed by your ALB
<code>groupRoles</code>	The mapping from Okta groups to Spring roles, e.g. <code>orders-editor</code> → <code>EDITOR</code>

Why it's a separate class: these values change between dev, UAT and prod. Keeping them in config means changing the ALB, region or role mapping doesn't need a code change.

AlbTokenVerifier: checks that the token is real

The ALB puts a signed JWT in the `x-amzn-oidc-data` header. This class answers one question: **"Was this token really issued by our ALB, and is it still valid?"** If it was, it returns the user's claims (email, groups and so on). If not, it throws an error.

It runs five checks, in order:

#	CHECK	WHAT IT PREVENTS
1	Algorithm is ES256	A forged token that claims a weaker or "none" algorithm
2	signer header equals our ALB ARN	A valid token from someone else's ALB. Every ALB in a region uses keys from the same endpoint, so without this check another AWS account's ALB could create tokens your service accepts.
3	kid format is valid	Using an attacker-supplied value in the key URL
4	Signature is valid	Tampered tokens, e.g. someone editing the payload to add <code>orders-editor</code>
5	Not expired	Replaying an old token that was captured earlier

It also **fetches and caches the ALB's public key**. The first token with a given key ID triggers an HTTPS call to AWS for that key. The key is kept in memory, so later requests don't repeat the call.

What it doesn't do: it knows nothing about roles or Spring Security. It only verifies the token. That keeps it simple to test and reusable elsewhere.

AlbAuthenticationFilter: turns the token into a Spring user

A servlet filter that runs **once per request, before the controller**. It connects "a header on the request" to "a logged-in user Spring Security knows about."

What it does on each request:

1. **Reads** the `x-amzn-oidc-data` header.
2. **Calls** `AlbTokenVerifier.verify()` to check it.
3. **Pulls out** the `groups` claim, e.g. `["orders-readonly"]`.
4. **Maps** each group to a role using `AlbProperties.groupRoles`: `orders-readonly` → `ROLE_READER`, `orders-editor` → `ROLE_EDITOR`, anything else is ignored.
5. **Stores** the user and their roles in Spring's `SecurityContext`. `@PreAuthorize` reads that context later.

How it handles each case:

SITUATION	RESULT
Valid token	User stored with roles; request continues
Invalid token (bad signature, wrong signer, expired)	Returns 401 immediately; controller never runs
No header	Passes the request on unauthenticated; SecurityConfig then returns 401

Why the `ROLE_` prefix: Spring's `hasRole('EDITOR')` looks for an authority named `ROLE_EDITOR`. The prefix is what makes `@PreAuthorize("hasRole('EDITOR')")` match.

SecurityConfig: connects everything and sets the rules

The configuration class. It doesn't process requests itself. It tells Spring Security **how** to process them.

SETTING	WHAT IT DOES
<code>@EnableMethodSecurity</code>	Makes <code>@PreAuthorize</code> on controller methods work. Without it, those annotations are silently ignored and every user can call every endpoint .
<code>@EnableConfigurationProperties</code>	Loads <code>application.yml</code> values into <code>AlbProperties</code>
<code>.addFilterBefore(...)</code>	Adds the filter to the chain. It runs before the anonymous filter, so a verified user is set before Spring falls back to treating the request as anonymous.
<code>STATELESS</code> sessions	No HTTP session in the service. Every request is checked from its own header, which suits the ALB setup.
<code>/actuator/health</code> → <code>permitAll()</code>	ALB health checks don't carry a token. Without this, every target would be marked unhealthy.
<code>anyRequest().authenticated()</code>	Everything else needs a verified user
<code>HttpStatusEntryPoint.UNAUTHORIZED)</code>	Returns a plain 401 instead of Spring's default login redirect, which would make no sense here
<code>csrf.disable()</code>	Keeps the example simple. Turn it back on if a browser UI calls the edit endpoints.

Who decides what

QUESTION	ANSWERED BY
Who is this user? (login)	ALB + Okta
Is this token real?	AlbTokenVerifier
What roles does this user have?	AlbAuthenticationFilter
Where do the settings come from?	AlbProperties
Which requests need a user, and how does the chain run?	SecurityConfig
Can this user call this specific endpoint?	<code>@PreAuthorize</code> on the controller

The main thing to take away is that **authentication** (checking identity) and **authorization** (checking permissions) happen in different places. The ALB and the verifier check identity, the filter turns groups into roles, and `@PreAuthorize` makes the final allow-or-deny decision.

Step 4: Validate it

4a. Automated tests (no ALB or Okta needed)

These tests replace the verifier with a mock, so they check the **real group-to-role mapping** and the `@PreAuthorize` rules.

```
package com.example.orders.api;

import com.example.orders.security.*;
import com.nimbusds.jwt.JWTClaimsSet;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.test.context.TestPropertySource;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.*;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(OrderController.class)
@Import(SecurityConfig.class)
@TestPropertySource(properties = {
    "alb.group-roles.orders-readonly=READER",
    "alb.group-roles.orders-editor=EDITOR"})
class OrderRbacTest {

    @Autowired MockMvc mvc;
    @MockitoBean AlbTokenVerifier verifier; // Spring Boot < 3.4: use @MockBean

    private static final String BODY = "{\"item\":\"Laptop\",\"quantity\":3}";

    private void userInGroups(String... groups) throws Exception {
        when(verifier.verify(any())).thenReturn(new JWTClaimsSet.Builder()
            .subject("00u123").claim("email", "user@example.com")
            .claim("groups", List.of(groups))
            .expirationTime(new Date(System.currentTimeMillis() + 60_000))
            .build());
    }

    @Test void reader_canRead() throws Exception {
        userInGroups("orders-readonly");
        mvc.perform(get("/api/orders").header("x-amzn-oidc-data", "t"))
            .andExpect(status().isOk());
    }

    @Test void reader_cannotEdit() throws Exception {
        userInGroups("orders-readonly");
        mvc.perform(put("/api/orders/1").header("x-amzn-oidc-data", "t")
            .contentType(MediaType.APPLICATION_JSON).content(BODY))
            .andExpect(status().isForbidden());
    }

    @Test void editor_canEdit() throws Exception {
        userInGroups("orders-editor");
    }
}
```

```

        mvc.perform(put("/api/orders/1").header("x-amzn-oidc-data", "t")
            .contentType(MediaType.APPLICATION_JSON).content(BODY))
            .andExpect(status().isOk());
    }

    @Test void unmappedGroup_forbidden() throws Exception {
        userInGroups("some-other-group");
        mvc.perform(get("/api/orders").header("x-amzn-oidc-data", "t"))
            .andExpect(status().isForbidden());
    }

    @Test void noToken_unauthorized() throws Exception {
        mvc.perform(get("/api/orders")).andExpect(status().isUnauthorized());
    }

    @Test void invalidToken_unauthorized() throws Exception {
        when(verifier.verify(any())).thenReturn(new BadCredentialsException("bad"));
        mvc.perform(get("/api/orders").header("x-amzn-oidc-data", "forged"))
            .andExpect(status().isUnauthorized());
    }
}

```

4b. End-to-end checks after deployment

Sign in with two test users: one in `orders-readonly` and one in `orders-editor`.

1. **Check the resolved roles.** Open `https://orders.example.com/api/me`. The reader should see `["ROLE_READER"]` and the editor `["ROLE_EDITOR"]`. If `roles` is empty, the `groups` claim isn't reaching the service. Check the Okta claim setup in Step 1c.
2. **Test the edit endpoint with curl.** Copy the `AWSSELBAuthSessionCookie-*` cookies from your browser's devtools, then:

```

curl -i -X PUT https://orders.example.com/api/orders/1 \
-H "Content-Type: application/json" \
-b "AWSSELBAuthSessionCookie-0=...; AWSSELBAuthSessionCookie-1=..." \
-d '{"item":"Laptop","quantity":3}'

```

SCENARIO	EXPECTED RESULT
Reader: GET <code>/api/orders</code>	200
Reader: PUT <code>/api/orders/1</code>	403
Editor: GET and PUT	200
User in no <code>orders-*</code> group	403 on all order endpoints
User not assigned to the Okta app	Blocked by Okta at login
Direct call to the instance with a fake <code>x-amzn-oidc-data</code> header	401 (signer or signature check fails)
API call through the ALB with no cookie (<code>deny</code> mode)	401 from the ALB

Gotchas

The signer check matters. Every ALB in a region signs with keys from the same public key endpoint. Without the `signer` check, a token from **someone else's ALB** would pass signature verification.

Accept traffic only from the ALB. The backend's security group should allow inbound traffic from the ALB's security group only. The signer check is your second layer of protection, not the only one.

Group changes aren't instant. The ALB stores the user's claims in its session cookie at login. If you remove someone from `orders-editor`, they may keep edit access until their ALB session ends. A short `session_timeout` limits that window.

CSRF. The ALB session is a browser cookie, so a malicious site could trigger a PUT using the user's cookie. The example disables Spring's CSRF protection to keep the code simple. If a browser UI calls the edit endpoints, turn CSRF protection back on, or require a custom header such as `X-Requested-With`.

Health checks. ALB target group health checks don't carry a token, so `/actuator/health` must stay `permitAll`. Otherwise every target is marked unhealthy.

Group names in YAML. Lowercase names with hyphens bind fine. If your Okta groups have uppercase letters, use bracket notation, e.g. `"[Orders-Editor]": EDITOR`, or Spring may change the key.